

Contents

Методы кластерного анализа. Методы выявления (выращивания) кластеров. . . .	1
Методы кластеризации:	1
Наиболее распространенные методы кластерного анализа можно условно разделить на две группы:	2
Односвязывающий метод	2
Полносвязывающий метод. Алгоритм максиминного расстояния	3
Метод k-средних (не нашел всех картинок, поэтому текст ниже основан на информации из открытых источников. Если у вас есть картинки — проверяйте, ничего не гарантирую.)	3
Алгоритм ISODATA	5
Алгоритмы кластеризации класса FOREL	7
Алгоритм CURE	9

Методы кластерного анализа. Методы выявления (выращивания) кластеров.

Методы кластеризации:

- 1. Графовые методы кластеризации.**
Наиболее примитивный класс алгоритмов. В настоящее время практически не применяется на практике.
- 2. Вероятностные алгоритмы кластеризации.**
Каждый объект из обучающей выборки относится к каждому из кластеров с определенной степенью вероятности.
- 3. EM-алгоритм.**
- 4. Иерархические методы кластеризации.**
Упорядочивание данных путем создания иерархии вложенных кластеров.
- 5. Алгоритм k-means (k-средних).**
Итеративный алгоритм, основанный на минимизации суммарного квадратичного отклонения точек кластеров от центров этих кластеров.
- 6. Распространение похожести (affinity propagation)**
Распространяет сообщения о похожести между парами объектов для выбора типичных представителей каждого класса.
- 7. Сдвиг среднего значения (mean shift).**
Выбирает центроиды кластеров в областях с наибольшей плотностью.
- 8. Спектральная кластеризация (spectral clustering).**
Использует собственные значения матрицы расстояний для понижения размерности перед использованием других методов кластеризации.
- 9. Основанная на плотности пространственная кластеризация для приложений с шумами (density-based spatial clustering of applications with noise, DBSCAN)**
Алгоритм группирует в один кластер точки в области с высокой плотностью. Одиноко расположенные точки помечает как шум.

Наиболее распространенные методы кластерного анализа можно условно разделить на две группы:

1. Методы выявления (выращивания) кластеров при заданном пороговом ограничении на расстояние между точками множества. В данном подходе количество кластеров, как правило, неизвестно. Исходными данными при такой постановке задачи считаются пороговое ограничение расстояния и правила объединения элементов множества. В результате количество и форма кластеров сильно зависит от выбранного метода анализа, величины порога и начальных условий.

По методам формирования кластеров в этом подходе выделяются: - **Односвязывающие методы** (анализ элементов, ближайших к текущему) - **Полносвязывающие методы** (анализ наиболее удаленных элементов) - **Среднесвязывающие методы**

2. Методы формирования кластеров при заданном количестве групп. Задается исходное количество центров кластеров, которые в процессе анализа перемещаются таким образом, чтобы заданные требования к кластерам выполнялись наилучшим образом. Как правило, здесь имеется критерий качества кластеризации, который в процессе формирования кластеров максимизируется (или минимизируется).

К этой группе алгоритмов относятся: - Алгоритмы К внутригрупповых средних - **ISODATA** (Iterative Self-Organizing Data Analysis Techniques) - Дисперсионно-ковариационные критерии оптимизации

Данные алгоритмы представляют наибольший интерес, далее рассмотрим их подробнее.

Примечание: И в том, и в другом подходах могут быть заданы дополнительные требования к кластерам: минимальное количество точек в группе, минимальное расстояние между группами и некоторые другие. Существует большое количество более или менее эффективных алгоритмов кластеризации, основанных на теории графов.

Односвязывающий метод

Простейший способ выявления кластеров:

1. Выбираем произвольную точку (вектор) множества X , назначаем её центром первого кластера ($x_k, k = 1$) и присоединяем к этому все точки, удовлетворяющие условию:
$$\|x_i - x_k\| < d,$$

где $\|\cdot\|$ — мера сходства между образами,
 d — заданное пороговое ограничение по этой мере.
2. Первая точка, для которой условие не выполняется, назначается центром следующего кластера. Далее для каждой точки вычисляется расстояние уже до двух центров, и она относится к тому кластеру, расстояние до которого меньше. Если расстояние до существующих кластеров больше заданного ограничения, образуется новый кластер.

Итог: Получаем некоторое разбиение на классы, вид которого сильно зависит от порядка просмотра образов, особенно в тех случаях, когда тенденция к образованию групп прослеживается слабо.

Недостаток: Образование так называемых “цепных” или “серпантинных” кластеров. Для образования компактных групп образов, близких к гиперсферам, удобнее использовать полносвязывающий метод.

Полносвязывающий метод. Алгоритм максиминного расстояния

В качестве исходного образа выберем некоторую “крайнюю” точку, например, с минимальными координатами в пространстве X . Назовем эту точку m_1 , она будет центром кластера K_1 .

В качестве второго центра K_2 выберем наиболее удаленную от нее точку по всему множеству образов. Определим пороговое значение $d = \frac{\|m_1 - m_2\|}{2}$.

Пошаговый алгоритм:

1. Вычисляем расстояния до центров m_1 и m_2 , то есть $\|x - m_1\|, \|x - m_2\|$ для всех x из нашего множества образов. Из каждой пары расстояний выбираем минимальное.
 2. Определяем максимальное значение $M = \max[\min(\|x - m_1\|, \|x - m_2\|)]$ по всему множеству образов. Пусть этому значению соответствует образ x_i . Если $M > d$, назначаем x_i центром кластера K_3 . В качестве новой пороговой меры d можно взять величину $d = M/2$ или половину среднего значения по всем минимальным расстояниям, рассчитанным на предыдущем шаге.
 3. Для всех x вычисляем минимальное из K расстояний до центров уже образованных кластеров:
 $\min\|x - m_k\|, k = 1, \dots, K$.
-

Метод k-средних (не нашел всех картинок, поэтому текст ниже основан на информации из открытых источников. Если у вас есть картинки — проверяйте, ничего не гарантирую.)

Алгоритм K-средних (K-means) — итеративный метод разбиения набора данных на K непересекающихся кластеров. Каждый кластер описывается центроидом — средней точкой среди всех объектов внутри данного кластера.

Основные шаги алгоритма:

1. **Инициализация:**
Выбирается нужное число кластеров K . Случайно или с помощью эвристик инициализируются K центроидов — начальные центры кластеров.

2. Присвоение точек кластерам:

Для каждой точки данных вычисляется расстояние (обычно Евклидово) до всех центроидов. Точка присваивается тому кластеру, расстояние до центра которого минимально.

3. Пересчет центроидов:

Для каждого кластера вычисляется новый центроид как среднее арифметическое всех точек, попавших в данный кластер. Формула для обновления центроида кластера C_k :

$$centroid_k = \frac{1}{|C_k|} \sum_{x \in C_k} x,$$

где $|C_k|$ — количество точек в кластере.

4. Проверка сходимости:

Если центроиды изменились незначительно (например, изменение оказалось ниже заданного порога) или достигнуто максимальное количество итераций — алгоритм завершается. Иначе — возврат к шагу 2.

Ключевые особенности и пояснения:

1. Цель алгоритма — минимизировать сумму квадратов расстояний от точек до центроидов их кластеров (within-cluster sum of squares, WCSS):

$$S = \sum_{k=1}^K \sum_{x \in C_k} \|x - centroid_k\|^2$$

K-means стремится найти локальный минимум этой функции.

2. Чувствительность к начальным центроидам

Разные начальные центроиды могут приводить к разным конечным кластерам. На практике часто алгоритм запускают несколько раз со случайной инициализацией и выбирают лучший результат по WCSS.

3. Предположения

Алгоритм предполагает, что кластеры:

- Имеют сферическую форму
- Одинаковы по размеру
- Разделимы (центроиды далеко друг от друга)

4. Сложность

Работает быстро на небольших данных. Сложность — $O(n \cdot K \cdot D \cdot i)$,

где:

n — число точек,

d — размерность,

i — число итераций.

5. Недостатки

- Требуется указания K заранее.
- Плохо работает с кластерами разного размера, плотности или сложной формы.
- Чувствителен к выбросам.

Аналогия от DeepSeek Представьте, что вы разбрасываете K флажков на карте (центроиды). Затем:

1. Все жители (точки) идут к ближайшему флажку — образуются деревни (кластеры).
 2. Флажки переносятся в центр своих деревень.
 3. Процесс повторяется, пока флажки не перестанут сдвигаться.
-

Алгоритм ISODATA

Алгоритм ISODATA (Iterative Self-Organizing Data Analysis Technique) — это итеративный алгоритм кластеризации, который является значительным развитием идеи К-средних.

Его ключевое отличие — **адаптивность**: алгоритм автоматически может менять количество кластеров K в процессе работы на основе структуры данных, что избавляет от главного недостатка K-means — необходимости жёстко задавать K заранее.

Основные шаги и логика алгоритма **Параметры алгоритма:** - K — желаемое количество кластеров. - θ_N — минимальное число точек в кластере (иначе кластер удаляется). - θ_S — максимальное стандартное отклонение (если кластер “расплывчатый” — делится). - θ_C — минимальное расстояние между центрами (если слишком близко — объединяются). - L — максимальное количество пар центров, которые можно объединить в блоке слияния. - I — максимальное число итераций.

Основной итеративный цикл:

1. **Инициализация и первое присваивание**
 - Задаются начальные центры (можно случайно, как в K-means). Количество может быть не равно целевому K .
 - Выполняется этап присваивания: все точки распределяются по ближайшему центру (как в K-means).
2. **Этап объединения (блок слияния, “Lumping”) — удаление мелких кластеров**
 - Если число точек в каком-либо кластере $N_k < \theta_N$, этот кластер ликвидируется. Его точки перераспределяются в другие кластеры. Данный шаг позволяет избавиться от кластеров-выбросов или артефактов.
3. **Пересчет центров кластеров**
 - Для каждого оставшегося кластера вычисляется новый центр — среднее всех его точек.
4. **Этап разделения (блок расщепления, “Splitting”) — контроль “компактности”**
 - Для каждого кластера вычисляется среднеквадратичное отклонение (стандартное отклонение) по каждой координате (признаку). Получается вектор отклонений σ_k .
 - **Условие разделения:** Если какая-либо компонента $\sigma_{k,d} > \theta_S$ и (текущее число кластеров $< K/2$ ИЛИ среднее внутрикластерное расстояние велико), то кластер считается некомпактным и разделяется.
 - **Разделение:** Создаются два новых центра, сдвинутые от старого центра вдоль координаты d на небольшое расстояние. Для этого выбирается некоторая величина $0 < \gamma < 1$. Новые центры получаются сдвигом старого центра на $\pm \gamma \sigma_{k,d}$. Величина γ выбирается так, чтобы новые кластеры были различимы, но не повлияли существенно на положение других кластеров.
5. **Этап объединения (слияние центров, Merging)**

- Для всех пар кластеров вычисляется расстояние между их центрами D_{ij} .
- **Условие объединения:** Если $D_{ij} < \theta_C$, то кластеры считаются слишком близкими.
- **Объединение:** Из всех таких пар выбирается самая близкая (или несколько, но не более L) и объединяется в один. Новый центр вычисляется как взвешенное среднее центров исходных кластеров (с учетом числа точек в каждом кластере).

6. Проверка условий остановки

- Цикл (шаги 1-5) повторяется.
- Алгоритм останавливается, если выполнено хотя бы одно:
 - Достигнуто максимальное число итераций I .
 - На двух последовательных итерациях число и текущая структура кластеров не меняются, т.е. достигнута стабильность.

Ключевые особенности и пояснения (DeepSeek)

- **Динамическое K:** Главная “фишка” ISODATA. Алгоритм не просто перемещает центры, а управляет их количеством на основе данных: разделяет крупные неоднородные кластеры и объединяет близкие мелкие.
- **Контроль качества кластеров:** Введены меры “компактности” (через θ_S) и “разделимости” (через θ_C). Это делает результат более осмысленным с точки зрения геометрии данных.
- **Чувствительность к параметрам:** Алгоритм очень гибкий, но эта гибкость — палка о двух концах. Результат сильно зависит от правильного выбора порогов ($\theta_N, \theta_S, \theta_C$), которые часто неочевидны и требуют настройки под конкретную задачу.
- **Вычислительная сложность:** Значительно выше, чем у K-means, из-за дополнительных этапов проверки и операций разделения/объединения на каждой итерации.
- **Применение:** Исторически и сейчас используется в задачах дистанционного зондирования Земли (классификация спутниковых снимков), где число природных классов (лес, вода, поле) заранее приблизительно известно, но их точные границы и количество вариаций — нет.

Сравнительная аналогия (тоже дипсик, пусть будет)

- **K-means:** У вас есть ровно 5 коробок ($K = 5$). Вы раскладываете предметы в ближайшую коробку, потом двигаете коробки в центр их содержимого. Процесс повторяется. В итоге все предметы в 5 коробках.
- **ISODATA:** Вы начинаете с примерно 5 коробок. После каждой “примерки” вы делаете инспекцию:
 1. **Объединение:** Если в коробке оказалось очень мало предметов — выкидываете её, а предметы раскидываете по другим.
 2. **Разделение:** Если в одной коробке предметы лежат слишком просторно и далеко друг от друга — берете две новые коробки и делите содержимое между ними.
 3. **Объединение:** Если две коробки стоят почти вплотную и их содержимое похоже — сливаете их в одну большую.

Вы повторяете раскладку и инспекцию, пока коробки не перестанут двигаться, а их количество не стабилизируется. Итоговое число коробок может стать и 3, и 7 — в зависимости от самих предметов.

Алгоритмы кластеризации класса FOREL

Про FOREL на имеющихся слайдах информации ноль. Ниже описание из открытых источников + помощь нейронок.

Алгоритм кластеризации FOREL (FORmal ELement — формальный элемент) — это итеративный алгоритм, основанный на идее “притягивания” точек внутрь гипертсферы заданного радиуса и последовательного “вычерпывания” кластеров из данных. Его часто называют алгоритмом “всплывающего шара” или “поиска сфер”.

Ключевая философия: Вместо поиска центров кластеров (как K-means) FOREL ищет плотные сгустки точек, “накрывая” их сферой фиксированного радиуса R .

Основные шаги алгоритма: **Входные параметры:** - R — радиус сферы (окна). Это самый важный параметр, определяющий масштаб поиска кластеров. - ϵ — точность (порог остановки для смещения центра сферы).

1. **Выбор начальной точки.**

Из множества некластеризованных точек случайным образом выбирается точка P .

2. **Формирование “ядра” кластера.**

Вокруг текущего центра сферы (изначально — точка P) определяется множество точек, попавших внутрь сферы радиуса R .

3. **Пересчёт центра тяжести “ядра”.**

Для всех точек, попавших в сферу, вычисляется их центр масс (среднее арифметическое по координатам). Этот новый центр становится центром сферы на следующем шаге.

4. **Итеративное уточнение центра сферы.**

Шаги 2 и 3 повторяются итеративно: на каждом шаге мы находим точки внутри сферы с текущим центром, затем вычисляем новый центр тяжести этих точек и перемещаем сферу туда.

Этот процесс продолжается, пока смещение центра сферы между итерациями не станет меньше порога ϵ . Сфера “застревает” в локальном сгустке точек.

5. **Фиксация кластера.**

Все точки, оказавшиеся внутри сферы в её финальном положении, образуют кластер. Эти точки исключаются из дальнейшей обработки (помечаются как кластеризованные).

6. **Переход к следующему кластеру.**

Из оставшихся некластеризованных точек снова выбирается случайная начальная точка, и процесс (шаги 1-5) повторяется.

Алгоритм завершается, когда все точки отнесены к какому-либо кластеру.

Ключевые особенности Преимущества: - Адаптивная форма кластера

В отличие от K-means, который навязывает сферическую форму *каждому кластеру целиком*, FOREL находит локальные сгустки, которые могут образовывать кластеры **произвольной формы**. Сфера R — это инструмент *поиска*, а не форма *результата*.

- **Устойчивость к начальному выбору**

Благодаря итеративному пересчету центра тяжести, алгоритм слабее зависит от выбора начальной точки. Сфера “сползает” в ближайший локальный максимум плотности.

Чувствительность к параметрам: - **Маленький R** — алгоритм найдет множество мелких, очень плотных кластеров (риск переобучения, выделения шума) - **Большой R** — сфера будет “проглатывать” несколько сгустков, объединяя их в один большой кластер (недообучение)

Вычислительные аспекты: - **Сложность:** Наивная реализация — $O(n^2)$, так как на каждой итерации нужно найти все точки внутри сферы - **Оптимизация:** На практике используют пространственные индексы (KD-деревья, R-деревья) для ускорения поиска точек в радиусе R

Наглядная аналогия Представьте, что у вас есть лист, усыпанный **железными опилками** (точки данных), и вы используете **магнит** определенной силы (радиус R):

1. **Подносите магнит** к случайному месту на листу (выбор начальной точки)
2. Опилки в радиусе действия магнита **прилипают к нему** (формирование ядра)
3. **Перемещаете магнит** в центр тяжести прилипшей группы опилок (пересчет центра)
4. Снова **смотрите, какие опилки теперь в радиусе**, и снова **сдвигаете магнит** в их центр масс. Повторяете, пока магнит не перестанет двигаться (итеративное уточнение)
5. Все опилки, которые в итоге примагнитились, **снимаете с листа и кладёте в отдельную кучку — это кластер №1** (фиксация кластера)
6. Берете магнит и **идёте к оставшимся на листе опилкам**, повторяя процесс, пока лист не опустеет

Итог: Размер магнита (R) критически важен. Слишком маленький магнит соберет только маленькие, плотные комочки. Слишком большой — соберет почти все опилки в одну большую кучу. Задача — подобрать магнит, соответствующий естественному размеру “комочков” в ваших данных.

Применение

- Исторически использовался в задачах **распознавания образов и анализа изображений**
- Эффективен для данных с **неявной кластерной структурой**, где кластеры имеют **разную плотность и форму**

Алгоритм CURE

Алгоритм CURE (Clustering Using REpresentatives) — это иерархический алгоритм агломеративной кластеризации, разработанный для эффективной работы с большими наборами данных и способный находить кластеры произвольной формы и размеров, устойчивый к выбросам.

Выполняет иерархическую кластеризацию с использованием набора определяющих точек для определения объекта в кластер.

Ключевая идея: Вместо того, чтобы представлять кластер одним центроидом, CURE представляет каждый кластер набором репрезентативных точек, которые захватывают его геометрию.

Назначение: Кластеризация очень больших наборов числовых данных.

Ограничения: - Эффективен для данных низкой размерности - Работает только на числовых данных

Достоинства: - Выполняет кластеризацию на высоком уровне даже при наличии выбросов - Выделяет кластеры сложной формы и различных размеров - Обладает линейно зависимыми требованиями к месту хранения данных и временную сложность для данных высокой размерности

Недостатки: - Есть необходимость в задании пороговых значений и количества кластеров

Описание алгоритма по шагам (как в презентации):

1. **Построение дерева кластеров**, состоящего из каждой строки входного набора данных.
2. **Формирование “кучи” в оперативной памяти**, расчет расстояния до ближайшего кластера (строки данных) для каждого кластера. При формировании кучи кластеры сортируются по возрастанию дистанции от кластера до ближайшего кластера. Расстояние между кластерами определяется по двум ближайшим элементам из соседних кластеров. Для определения расстояния между кластерами используются “манхеттенская”, “евклидова” метрики или похожие на них функции.

3. Слияние ближних кластеров в один:

- Новый кластер получает все точки входящих в него входных данных
 - Расчет расстояния до остальных кластеров для новообразованного кластера.
- Для расчета расстояния кластеры делятся на две группы:

- **Первая группа:** кластеры, у которых ближайшими кластерами считаются кластеры, входящие в новообразованный кластер
- **Вторая группа:** остальные кластеры

При этом для кластеров из первой группы, если расстояние до новообразованного кластера меньше чем до предыдущего ближайшего кластера, то ближайший кластер меняется на новообразованный кластер.

- В противном случае ищется новый ближайший кластер, но при этом не берутся кластеры, расстояния до которых больше, чем до новообразованного кластера. Для кластеров второй группы выполняется следующее: если расстояние до

новообразованного кластера ближе, чем предыдущий ближайший кластер, то ближайший кластер меняется. В противном случае ничего не происходит.

4. **Переход на шаг 3**, если не получено требуемое количество кластеров.

Альтернативное описание (аналогов предыдущего описания не нашел, нейронки тоже его бракуют)

1. **(Для больших данных)** берут случайную выборку и/или делят данные на части, чтобы строить иерархию не на всём наборе сразу.
2. **Инициализация:** каждая точка (в выборке) — отдельный кластер.
3. Для каждого кластера выбирают фиксированное число s репрезентативных точек (representatives), отражающих форму кластера, и считают центроид μ .
4. **Shrinking:** каждую репрезентативную точку x сдвигают к центроиду на долю α :
$$x' = x + \alpha(\mu - x), 0 < \alpha < 1.$$
5. Расстояние между двумя кластерами определяют как минимум расстояний между их сжатыми репрезентативными точками.
6. Агломеративно сливают два ближайших кластера по этому расстоянию; для нового кластера заново выбирают representatives и выполняют shrinking.
7. Повторяют, пока не останется заданное число кластеров k ; затем (если была выборка) остальные точки приписывают ближайшему кластеру.